

Архитектура AI-поиска и чат-ассистента для корпоративной MediaWiki

1. Общая картина

Система объединяет корпоративную MediaWiki, аутентифицированную через Active Directory, с AI-поиском по содержимому вики. Поиск учитывает права доступа пользователя и не показывает документы, которые ему не положены. Вся система работает в замкнутом контуре без облачных сервисов.

Ключевое требование к безопасности: приемлемо, если пользователь временно видит документ, к которому потерял доступ; неприемлемо, если пользователь увидел документ, к которому никогда доступа не имел. Вся архитектура выстроена вокруг этого принципа.

2. Компоненты системы

Источник контента и аутентификации

MediaWiki — основная вики-платформа. Через расширения PluggableAuth, LDAPProvider, LDAPAuthentication2 и LDAPGroups подключена к Active Directory. Пользователи входят по корпоративным учётным данным, группы синхронизируются из AD в момент входа.

Active Directory — единственный источник групп пользователей. Вложенные группы разворачиваются через transitive membership.

Слой индексации

Syncer — сервис индексации. Получает события об изменениях страниц через хуки MediaWiki. При получении события парсит страницу, разбивает на фрагменты, получает эмбединги через локальную модель, и записывает в векторную базу. Для каждого фрагмента определяет ACL на уровне пространства имён и, если возможно, на уровне страницы.

Qdrant — векторная база данных. Хранит фрагменты текста в виде векторов и метаданных. Ключевое поле в метаданных — список групп, которым доступен фрагмент. Полнота индексации раз в сутки служит подстраховкой на случай потери событий.

AI-слой

AI Gateway — основной бэкенд поиска и чата. Живёт за reverse proxy на том же домене, что и MediaWiki. Получает запросы от пользовательского интерфейса, аутентифицирует через сессию MediaWiki, выполняет поиск в Qdrant с фильтрацией по группам, собирает контекст и запрашивает генерацию ответа.

LiteLLM Proxy — прокси-слой между Gateway и LLM-сервисами. Предоставляет единый OpenAI-совместимый endpoint. Распределяет нагрузку между Ollama и vLLM, обеспечивает отказоустойчивость, ограничение частоты запросов, кеширование и логирование.

Ollama и vLLM — локальные движки inference. Ollama используется для эмбедингов и лёгких моделей; vLLM — для production-нагрузки на большие языковые модели. Оба доступны только из внутренней сети.

Вспомогательные сервисы

Redis — кеш для групп пользователей. Gateway кеширует список групп из MediaWiki API на несколько минут, чтобы не нагружать вики при каждом запросе.

Nginx — reverse proxy. Все запросы приходят на один домен. Запросы к обычным страницам направляются в MediaWiki, запросы к поиску и чату — в AI Gateway. Cookie сессии MediaWiki таким образом автоматически доходят до Gateway.

3. Поток данных при поиске

Пользователь открывает в вики специальную страницу AI-поиска. На странице загружается JavaScript-приложение через систему ResourceLoader MediaWiki. Приложение делает запрос к AI Gateway по относительному пути.

Gateway извлекает cookie сессии из запроса и обращается к MediaWiki API с этой же cookie, чтобы узнать, кто пользователь и в каких он группах. Результат кладёт в Redis на короткое время.

Затем Gateway отправляет текст запроса в локальную модель эмбедингов через Ollama и получает вектор. С этим вектором и со списком групп пользователя обращается к Qdrant. Qdrant ищет семантически близкие фрагменты, но только среди тех, чей список разрешённых групп пересекается с группами пользователя.

Найденные фрагменты проходят через Gateway. Если фрагмент имеет простые ACL уровня пространства имён — он проходит сразу. Если фрагмент имеет индивидуальные ограничения или помечен как требующий проверки —

Gateway делает дополнительный запрос к MediaWiki API, чтобы убедиться, что конкретный пользователь может читать конкретную страницу. Если проверка не проходит или MediaWiki недоступна — фрагмент отбрасывается.

Прошедшие проверку фрагменты собираются в контекст. Gateway формирует промпт из вопроса пользователя и контекста документов, и отправляет запрос на генерацию в LiteLLM Proxy. LiteLLM выбирает между доступными LLM-сервисами, применяет rate limiting, и при необходимости переключается на резервный движок.

Ответ от LLM возвращается в Gateway в виде потока токенов. Gateway через Server-Sent Events передаёт токены в браузер пользователя, который видит, как текст появляется построчно. Вместе с ответом Gateway возвращает список источников — страниц вики, на которых основан ответ.

4. Поток данных при чате

Чат работает аналогично поиску, но с добавлением истории диалога. Gateway хранит историю сообщений в Redis привязанной к сессии пользователя. При каждом новом сообщении история дополняется и передаётся в LLM вместе с релевантными фрагментами из вики.

Контекст из вики подбирается для каждого сообщения отдельно — история диалога не влияет на ACL, каждый поиск выполняется с фильтрацией по группам пользователя. LiteLLM Proxy логирует все чат-запросы для возможности аудита.

5. Модель прав доступа

Принцип запрета при сомнении

Если Gateway не может однозначно подтвердить, что пользователь имеет право видеть документ — документ не показывается. Это приводит к тому, что иногда пользователь получает меньше результатов, чем мог бы, но исключает информационные утечки.

Уровни ACL

Первый уровень — пространства имён. Большинство корпоративных вики структурируют доступ по отделам и проектам через пространства имён. Syncer надёжно определяет ACL на этом уровне, потому что пространства настраиваются централизованно.

Второй уровень — индивидуальная защита страниц. Если страница имеет явные ограничения и Syncer может их однозначно интерпретировать — они

индексируются. Если ограничения сложные или зависят от внешних факторов — страница индексируется консервативно, с более узким набором групп, или помечается для обязательной проверки Gateway.

Третий уровень — runtime-проверка. Gateway для сомнительных результатов спрашивает MediaWiki, может ли пользователь читать эту страницу. Это ловит случаи, когда индекс устарел или когда права страницы слишком сложны для предвычисления.

Задержка обновления прав

Если администратор AD удалил пользователя из группы — пользователь сохранит доступ до перелога или истечения кеша. Это stale access, приемлемый в данной модели. Если пользователя добавили в группу — он получит доступ после перелога. Это задержка доступности, также приемлемая. Обратный сценарий — показать документ тому, кто никогда не имел доступа — архитектурно исключён.

6. Размещение кода

Тонкий слой в MediaWiki

PHP-расширение минимального размера. Регистрирует специальную страницу AI-поиска и чата. Через ResourceLoader загружает JavaScript-приложение. Реализует хуки для отправки событий в Syncer при редактировании, удалении, переименовании и изменении защиты страницы. Весь PHP-код занимает сотню строк — он не выполняет сложной логики, а лишь предоставляет точку входа и механизм уведомлений.

AI Gateway

Отдельный сервис на Node.js или Python. Содержит всю бизнес-логику: аутентификацию через MediaWiki API, взаимодействие с Qdrant, формирование промптов, стриминг ответов через Server-Sent Events, кеширование групп в Redis, управление историей чата. Выбор языка определяется командой — оба языка справляются с задачей, Node.js имеет преимущество в стриминге и единстве с фронтендом.

LiteLLM Proxy

Отдельный сервис, обычно в Docker-контейнере. Конфигурируется через файл, в котором описаны upstream-движки Ollama и vLLM. Gateway обращается к LiteLLM, а не напрямую к движкам. Это позволяет менять конфигурацию LLM без изменения кода Gateway.

Syncer

Сервис индексации, работающий в фоне. Получает webhook-и от MediaWiki, выполняет полную переиндексацию по расписанию как подстраховку. Взаимодействует с MediaWiki API для получения содержимого страниц, с Ollama для эмбедингов, с Qdrant для записи.

7. Инфраструктура и деплой

Все компоненты разворачиваются в замкнутом контуре. MediaWiki и AI Gateway находятся за одним reverse proxy на одном домене — это решает проблемы CORS и передачи сессий.

LLM-сервисы и Qdrant доступны только из внутренней сети, не имеют публичных endpoint. LiteLLM Proxy служит единой точкой входа для LLM-запросов и может применять rate limiting, чтобы предотвратить перегрузку локальных GPU.

Резервирование между Ollama и vLLM обеспечивается на уровне LiteLLM: если один движок недоступен, запросы автоматически направляются к другому. Это критично для production, где недоступность LLM блокирует работу пользователей.

8. Роль LiteLLM в архитектуре

LiteLLM выполняет несколько функций, которые иначе пришлось бы реализовывать самостоятельно в Gateway.

Унификация API. Ollama и vLLM имеют разные форматы запросов. LiteLLM предоставляет единый OpenAI-совместимый интерфейс. Gateway пишет код один раз, а подключение нового движка требует только изменения конфигурации LiteLLM.

Load balancing и fallback. Если vLLM перегружен или упал, LiteLLM перенаправляет запрос в Ollama. Это прозрачно для Gateway и пользователя.

Rate limiting. LiteLLM ограничивает количество запросов в минуту и токенов в час. Это защищает локальные GPU от перегрузки и позволяет справедливо распределять ресурсы между пользователями.

Кеширование. LiteLLM может кешировать ответы на идентичные запросы. Для корпоративной вики, где сотрудники задают похожие вопросы, это снижает нагрузку на GPU.

Логирование и аудит. LiteLLM записывает все запросы и ответы. В корпоративной среде это важно для аудита — можно проверить, кто и о чём спрашивал AI-ассистента.

Виртуальные ключи. Если в будущем потребуется дать разным сервисам разный уровень доступа к LLM, LiteLLM позволяет выпускать виртуальные ключи с разными ограничениями, не изменяя конфигурацию движков.

9. Границы ответственности

Компонент	За что отвечает	За что не отвечает
MediaWiki	Контент, аутентификация через AD, ACL страниц, сессии	AI-поиск, LLM, векторная база
AD/LDAP Stack	Группы пользователей	Контент, поиск
Syncer	Индексация контента, эмбединги, ACL в Qdrant	Аутентификация, ответы пользователю
Qdrant	Векторный поиск с фильтрацией	Проверка прав, генерация текста
AI Gateway	Аутентификация запросов, поиск, формирование промптов, стриминг	Хранение контента, LDAP
LiteLLM	Routing к LLM, rate limiting, кеш, логи	Бизнес-логика, ACL
Ollama/vLLM	Генерация текста, эмбединги	Поиск по вики, права доступа

Эта архитектура закрывает все обсуждавшиеся требования: self-hosted размещение, работа без облаков, интеграция с Active Directory, учёт прав доступа при поиске, защита от утечек, масштабируемость и отказоустойчивость LLM-слоя.